

Part D: Proving the Classifier Is Production-Ready

1. Building the eval set

I wouldn't start from synthetic data. The most useful examples are the messages guests already send us, so the first batch comes from our chat logs: a few weeks of WhatsApp and in-app conversations, sampled across days and hours so it isn't all weekday-morning traffic. Names, phone numbers and booking codes are masked before anyone labels them.

The labelling is done by people from the CS team, because they know what a message like "unit yang kemarin gimana" usually refers to. They get a one-page guideline with two or three examples per intent. About 15% of the messages are labelled by two people. When two labellers disagree, it's usually the guideline that's unclear rather than someone being careless, so we fix the guideline and relabel those items. Each message is also labelled with its entities (stays, dates, unit type) and the date it was sent. Without that date there's no way to check whether "besok" was resolved correctly.

For size, around 1,500 messages is a reasonable first version. Roughly 1,000 of them are a plain random sample, so the set keeps the real mix of traffic, and the rest are added on purpose to cover the gaps. My rule of thumb is at least 100 examples per intent and never fewer than 50. With 100 examples and a recall around 90%, the confidence interval is roughly plus or minus 6 points, which is enough to notice a real 10-point drop. Below 50 the numbers move around too much to trust.

Rare intents need extra effort, because a random sample barely contains them. Maintenance requests or injection attempts may be only 2 or 3% of traffic. I'd go looking for them: keyword searches ("bocor", "mati", "rusak", "refund"), similarity search around a handful of known examples, and tickets that CS moved to the maintenance team. On top of that, a small set of hand-written cases covers things that are rare but must not break: prompt injection, one message with two requests, "15 Maret" sent in September, "lusa", messages that are only an emoji. Those are marked as synthetic and stay a minority of the set.

The set is split once. About 40% is used day to day while working on the prompt. The other 60% is frozen and only used to decide whether a change ships, so we don't slowly tune the prompt to the test itself. The set is versioned, and every month or quarter it gets fresh production samples, especially messages the model got wrong or that CS rerouted. The five messages from the sample inbox, with the expected outputs from Part A, stay in a small must-pass set.

2. What to measure, and why accuracy is misleading here

Overall accuracy is the wrong headline number, for two reasons: the classes are very unbalanced, and the mistakes don't cost the same. If 55% of messages are booking questions and 8% are maintenance, a model that never recognises maintenance can still score above 90%. But sending "AC bocor parah" to the sales queue is far worse than asking a person to look at a booking question that didn't need it. Accuracy on the intent also ignores the dates, and the right intent with the wrong check-in date is still a wrong booking.

What I'd look at instead:

- Precision and recall for each intent, and macro-F1 so every class counts equally. The confusion matrix is where the useful detail is, for example booking vs extension, or payment vs unknown.

- Recall on the messages that must reach a person: high urgency, injection attempts, angry guests. This gets a hard floor, around 98%, because missing one of these is the expensive mistake.
- The escalation rate itself. Escalating everything is safe but defeats the purpose, so we watch both sides of that trade-off.
- Entity accuracy per field: exact match on each date (explicit dates and relative ones reported separately), location, unit type and the number of stays.
- Pipeline numbers: how often the output is invalid JSON, the retry and timeout rates, p95 latency, and cost per thousand messages.
- Whether confidence is calibrated. The service escalates anything below 0.6, and that threshold only makes sense if answers given with 0.6 confidence are right about 60% of the time. We check this with a reliability curve on the dev split and set the threshold there.

3. Knowing a change didn't make it worse

The prompt, the model and its parameters, the output schema and the guardrail thresholds are treated as one versioned unit, and that version is saved on every classification record. Whenever any of them changes:

1. CI runs the new version and the current one on the frozen split, at temperature 0, a few times each to see how much the output varies on its own.
2. The two versions are compared on the same messages, not only on averages. The change passes only if:
 - recall on maintenance, must-escalate and injection cases doesn't drop by more than a couple of points
 - macro-F1 isn't worse beyond noise (checked with a paired bootstrap, or McNemar on the messages whose label changed)
 - invalid output stays under 0.5%
 - latency and cost stay in budget
 - the must-pass set is 100% correct
3. Someone reads every message whose label flipped. The averages can look fine while the new prompt has picked up one specific new mistake.
4. The new version runs in shadow on live traffic for a few days: it classifies everything, but its answers aren't used. We compare it with the current version and with what CS actually did with those messages.
5. The rollout is gradual (5%, then 25%, then everyone), with an automatic rollback tied to the monitoring below.

4. Seeing it degrade before a guest complains

Most of these signals come almost for free from what the Part B service already logs and stores.

The first group is pipeline health: invalid-output rate, retries, timeouts, 502 and 504 responses, and latency. A sudden jump usually means an incident at the provider or a quiet model update on their side, which is also why the model version should be pinned.

The second group is about what the model is answering. The intent mix is compared with the same hour and weekday in previous weeks. If "unknown" doubles, or maintenance requests suddenly halve, something has changed even though nothing threw an error. The same comparison applies to:

- the confidence distribution
- the needs-human rate, broken down by reason
- how often booking questions come back without any dates
- how often the model returns an intent that isn't in the list

The third group watches the input. A new channel, a shift in the language mix or noticeably longer messages all mean we're receiving traffic the eval set doesn't cover yet.

The most honest signal is what happens after routing. If CS reroutes more auto-routed tickets than usual, or guests write "halo??" again within ten minutes, the classifier is probably getting things wrong. Next to that, someone labels around 200 production messages every week, which gives a real precision and recall trend instead of proxies.

Everything is broken down by prompt version, model version, channel and language, so when a number moves we can tie it to a specific change.